七、设计规约

1. 【强制】存储方案和底层数据结构的设计获得评审一致通过，并沉淀成为文档。
   说明：有缺陷的底层数据结构容易导致系统风险上升，可扩展性下降，重构成本也会因历史数据迁移和系统平滑过渡而陡然增加，所以，存储方案和数据结构需要认真地进行设计和评审，生产环境提交执行后，需要进行 double check。
   正例：评审内容包括存储介质选型、表结构设计能否满足技术方案、存取性能和存储空间能否满足业务发展、表或字段之间的辩证关系、字段名称、字段类型、索引等；数据结构变更（如在原有表中新增字段）也需要在评审通过后上线。

2. 【强制】在需求分析阶段，如果与系统交互的 User 超过一类并且相关的 UseCase 超过 5 个，使用用例图来表达更加清晰的结构化需求。

3. 【强制】如果某个业务对象的状态超过 3 个，使用状态图来表达并且明确状态变化的各个触发条件。
   说明：状态图的核心是对象状态，首先明确对象有多少种状态，然后明确两两状态之间是否存在直接转换关系，再明确触发状态转换的条件是什么。
   正例：淘宝订单状态有已下单、待付款、已付款、待发货、已发货、已收货等。比如已下单与已收货这两种状态之间是不可能有直接转换关系的。

4. 【强制】如果系统中某个功能的调用链路上的涉及对象超过 3 个，使用时序图来表达并且明确各调用环节的输入与输出。
   说明：时序图反映了一系列对象间的交互与协作关系，清晰立体地反映系统的调用纵深链路。

5. 【强制】如果系统中模型类超过 5 个，且存在复杂的依赖关系，使用类图来表达并且明确类之间的关系。
   说明：类图像建筑领域的施工图，如果搭平房，可能不需要，但如果建造蚂蚁 Z 空间大楼，肯定需要详细的施工图。

6. 【强制】如果系统中超过 2 个对象之间存在协作关系，并需要表示复杂的处理流程，使用活动图来表示。
   说明：活动图是流程图的扩展，增加了能够体现协作关系的对象泳道，支持表示并发等。

7. 【强制】系统设计时要准确识别出弱依赖，并针对性地设计降级和应急预案，保证核心系统正常可用。
   说明：系统依赖的第三方服务被降级或屏蔽后，依然不会影响主干流程继续进行，仅影响信息展示、或消息通知等非关键功能，那么这些服务称为弱依赖。
   正例：当系统弱依赖于多个外部服务时，如果下游服务耗时过长，则会严重影响当前调用者，必须采取相应降级措施，比如，当调用链路中某个下游服务调用的平均响应时间或错误率超过阈值时，系统自动进行降级或熔断操作，屏蔽弱依赖负面影响，保护当前系统主干功能可用。
   反例：某个疫情相关的二维码出错：“服务器开了点小差，请稍后重试”，不可用时长持续很久，引起社会高度关注，原因可能为调用的外部依赖服务 RT 过高而导致系统假死，而在显示端没有做降级预案，只能直接抛错给用户。

8. 【推荐】系统架构设计时明确以下目标：
   - 确定系统边界。确定系统在技术层面上的做与不做。
   - 确定系统内模块之间的关系。确定模块之间的依赖关系及模块的宏观输入与输出。
   - 确定指导后续设计与演化的原则。使后续的子系统或模块设计在一个既定的框架内和技术方向上继续演化。
   - 确定非功能性需求。非功能性需求是指安全性、可用性、可扩展性等。

9. 【推荐】需求分析与系统设计在考虑主干功能的同时，需要充分评估异常流程与业务边界。

10. 【推荐】类在设计与实现时要符合单一原则。
    说明：单一原则最易理解却是最难实现的一条规则，随着系统演进，很多时候，忘记了类设计的初衷。

11. 【推荐】谨慎使用继承的方式来进行扩展，优先使用聚合/组合的方式来实现。
    说明：不得已使用继承的话，必须符合里氏代换原则，此原则说父类能够出现的地方子类一定能够出现，比如，“把钱交出来”，钱的子类美元、欧元、人民币等都可以出现。

12. 【推荐】系统设计阶段，根据依赖倒置原则，尽量依赖抽象类与接口，有利于扩展与维护。
    说明：低层次模块依赖于高层次模块的抽象，方便系统间的解耦。

13. 【推荐】系统设计阶段，注意对扩展开放，对修改闭合。
说明：极端情况下，交付的代码是不可修改的，同一业务域内的需求变化，通过模块或类的扩展来实现。

14. 【推荐】系统设计阶段，共性业务或公共行为抽取出来公共模块、公共配置、公共类、公共方法等，在系统中不出现重复代码的情况，即 DRY 原则（Don't Repeat Yourself）。
说明：随着代码的重复次数不断增加，维护成本指数级上升。随意复制和粘贴代码，必然会导致代码的重复，在维护代码时，需要修改所有的副本，容易遗漏。必要时抽取共性方法，或者抽象公共类，甚至是组件化。
正例：一个类中有多个 public 方法，都需要进行数行相同的参数校验操作，这个时候请抽取：
private boolean checkParam(DTO dto) {...}

15. 【推荐】避免如下误解：敏捷开发=讲故事+编码+发布。
说明：敏捷开发是快速交付迭代可用的系统，省略多余的设计方案，摒弃传统的审批流程，但核心关键点上的必要设计和文档沉淀是需要的。
反例：某团队为了业务快速发展，敏捷成了产品经理催进度的借口，系统中均是勉强能运行但像面条一样的代码，可维护性和可扩展性极差，一年之后，不得不进行大规模重构，得不偿失。

16. 【参考】设计文档的作用是明确需求、理顺逻辑、后期维护，次要目的用于指导编码。
说明：避免为了设计而设计，系统设计文档有助于后期的系统维护和重构，所以设计结果需要进行分类归档保存。

17. 【参考】可扩展性的本质是找到系统的变化点，并隔离变化点。
说明：世间众多设计模式其实就是一种设计模式即隔离变化点的模式。
正例：极致扩展性的标志，就是需求的新增，不会在原有代码交付物上进行任何形式的修改。

18. 【参考】设计的本质就是识别和表达系统难点。
说明：识别和表达完全是两回事，很多人错误地认为识别到系统难点在哪里，表达只是自然而然的事情，但是大家在设计评审中经常出现语焉不详，甚至是词不达意的情况。准确地表达系统难点需要具备如下能力：表达规则和表达工具的熟练性。抽象思维和总结能力的局限性。基础知识体系的完备性。深入浅出的生动表达力。

19. 【参考】代码即文档的观点是错误的，清晰的代码只是文档的某个片断，而不是全部。
说明：代码的深度调用，模块层面上的依赖关系网，业务场景逻辑，非功能性需求等问题要相应的文档来完整地呈现。

20. 【参考】在做无障碍产品设计时，需要考虑到：
- 所有可交互的控件元素必须能被 tab 键聚焦，并且焦点顺序需符合自然操作逻辑。
- 用于登录校验和请求拦截的验证码均需提供图形验证以外的其它方式。
- 自定义的控件类型需明确交互方式。

正例：登录场景中，输入框的按钮都需要考虑 tab 键聚焦，符合自然逻辑的操作顺序如下，"输入用户名，输入密码，输入验证码，点击登录"，其中验证码实现语音验证方式。如有自定义标签实现的控件设置控件类型可使用 role 属性。